Recurrent Neural Networks and LSTMs: Theory, Architecture, Gradients, and Examples

Instructor: Prof. O. Olawale Awe

Why Recurrent Neural Networks?

RNNs are designed for **sequential or temporal data**, such as:

- Natural language (text, speech)
- Financial and scientific time series
- Sensor data and signals
- Any task with temporal dependencies

Traditional feedforward networks assume independence. RNNs incorporate **memory**:

$$h_t = f(h_{t-1}, x_t)$$

This enables learning patterns over time.

RNN Architecture (Unrolled Over Time)

RNN hidden state update:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

Output:

$$y_t = W_y h_t + b_y$$

Key characteristics:

- Same weights reused at each time step
- Hidden state stores past information
- Suitable for variable-length sequences

Forward Pass Equations

Hidden State Update:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h)$$

Output:

$$y_t = W_y h_t + b_y$$

Initial Condition:

$$h_0 = 0 \quad \text{(or learned parameter)}$$

Notes:

- tanh bounds activations in $[-1, 1]$
- Hidden state is the only memory mechanism
- Saturation of tanh $\rightarrow$ gradient problems

Why Backpropagation Through Time (BPTT)?

Because each hidden state depends on previous ones:

$$h_t \to h_{t-1} \to h_{t-2} \to \dots$$

To compute gradients, the RNN is **unrolled across time**.
Gradient w.r.t recurrent weight:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_h}$$

And:

$$\frac{\partial h_t}{\partial h_{t-1}} = W_h^\top \mathrm{diag}(1 - h_{t-1}^2)$$

This repeated multiplication causes instability.

Vanishing and Exploding Gradients

Backprop terms include:

$$\prod_{k=1}^{t} W_h^\top \mathrm{diag}(1 - h_k^2)$$

Two problems occur:
Vanishing gradients:

$$\|W_h\| < 1 \Rightarrow \text{product} \to 0$$

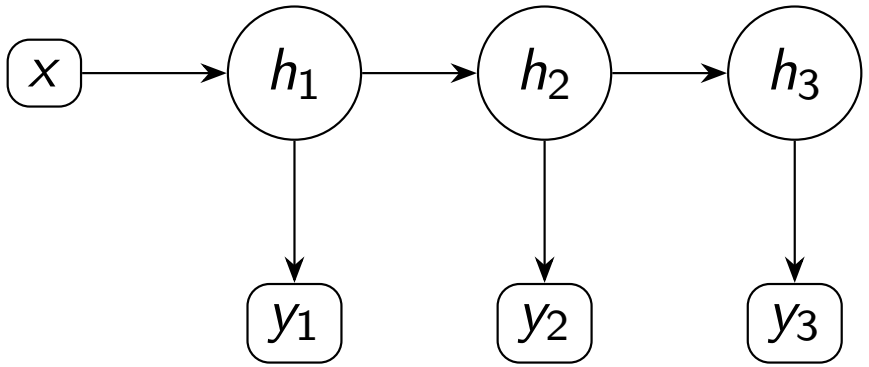
$\to$ RNN forgets long-term information.
Exploding gradients:

$$\|W_h\| > 1 \Rightarrow \text{product} \to \infty$$

$\to$ Training becomes unstable.
Key Idea: Vanilla RNNs fail on long sequences.

One-to-Many (Sequence Generation)

$$x \rightarrow (y_1, y_2, \ldots, y_T)$$

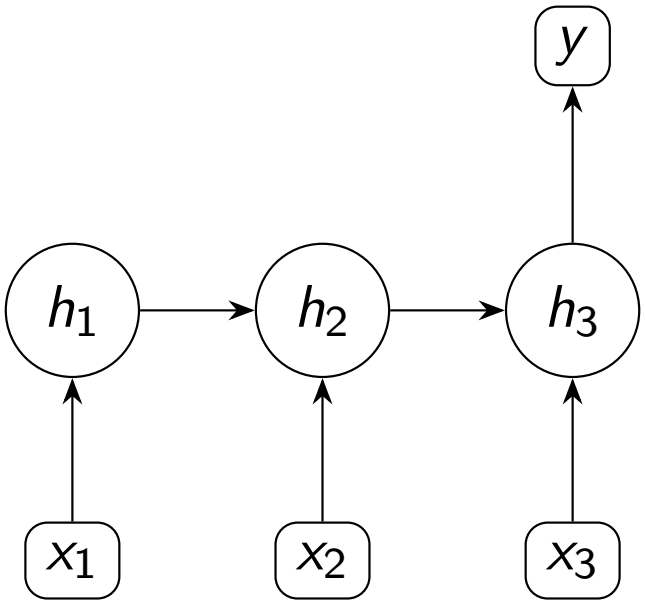


Many-to-One (Sequence Classification)

$$(x_1, x_2, \ldots, x_T) \rightarrow y$$

Examples:

- Sentiment analysis
- Spam detection
- Sequence-based risk scoring



Many-to-Many (Aligned Labeling)

$$x_t \rightarrow y_t$$

Examples:

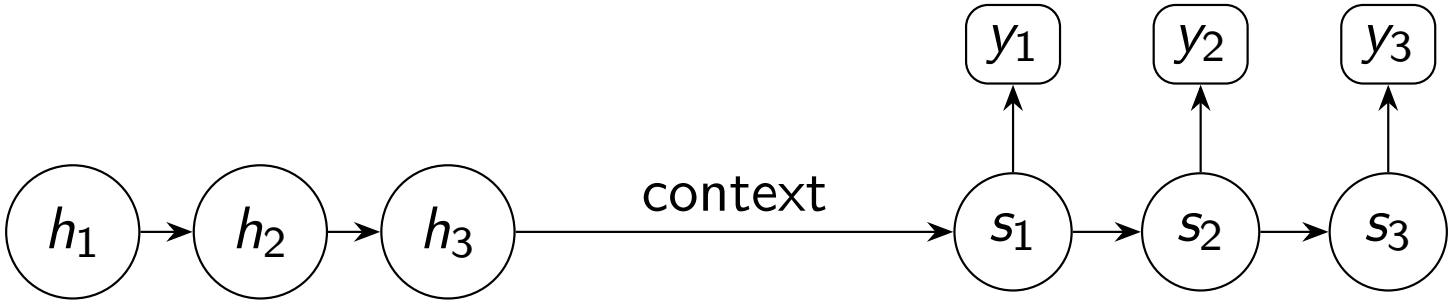
- POS tagging
- Named Entity Recognition (NER)
- Time-series labeling at each step

Many-to-Many (Encoder–Decoder)

$$(x_1, \ldots, x_T) \rightarrow (y_1, \ldots, y_{T'})$$

Examples:

- Machine translation
- Speech-to-text
- Summarization



Example: One RNN Step

Given:

$$x_t = 1, \quad h_{t-1} = 0.5, \quad W_x = 2, \; W_h = 0.6, \; b_h = 0$$

Compute:

$$h_t = \tanh(2(1) + 0.6(0.5)) = \tanh(2.3) \approx 0.980$$

Interpretation: $\tanh(2.3)$ is near saturation $\rightarrow$ derivative $0.04 \rightarrow$ vanishing gradient.

Why Do We Need LSTMs?

Vanilla RNNs suffer from:

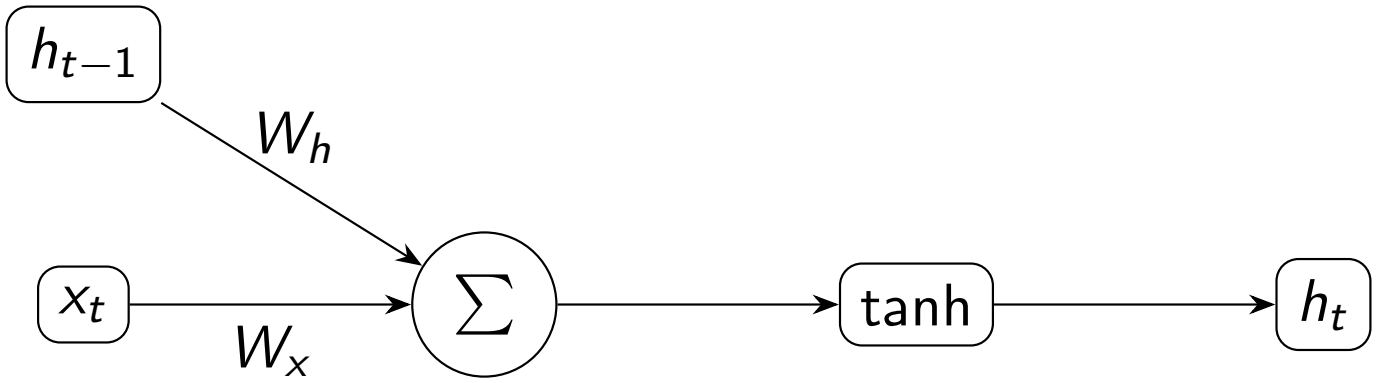
- Vanishing gradients
- Exploding gradients
- Inability to learn long-term dependencies

LSTMs introduce:

- A memory cell c_t (long-term memory)
- Gates that control information flow
- Additive updates (not multiplicative)
- The Constant Error Carousel (CEC)

This solves gradient decay over time.

Vanilla RNN Cell



$$h_t = \tanh(W_x x_t + W_h h_{t-1})$$

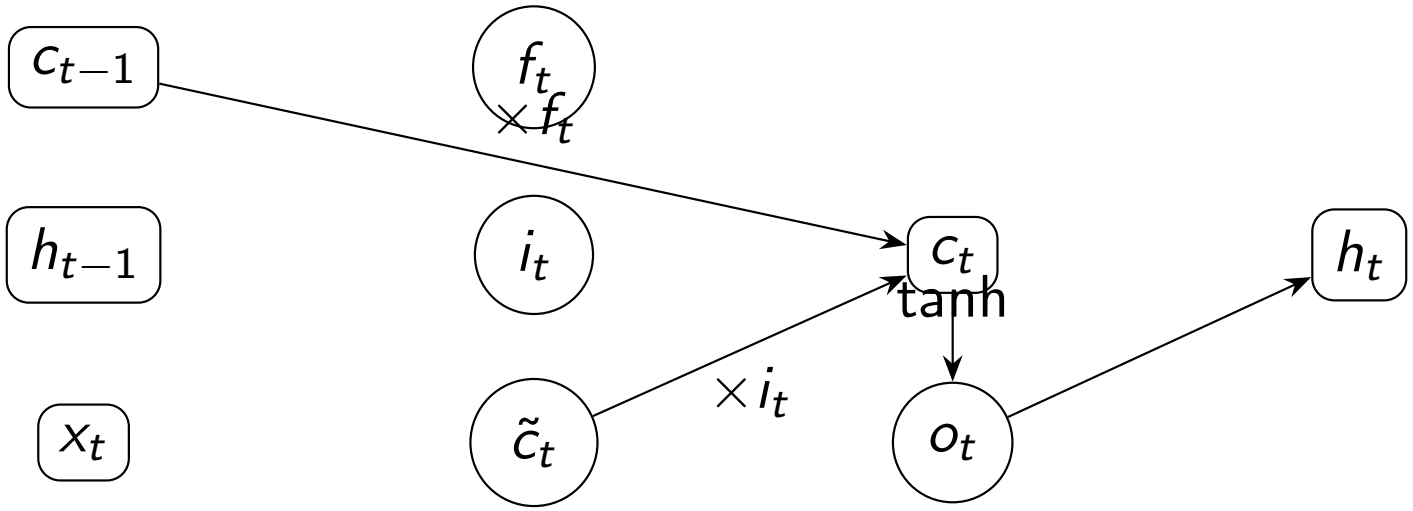
LSTM Cell

```
                +-------------------------------+
x_t -------> |            LSTM Cell            | ---> h_t
h_{t-1} ---> |                                 |
c_{t-1} ---> |                                 | ---> c_t
                +-------------------------------+
```

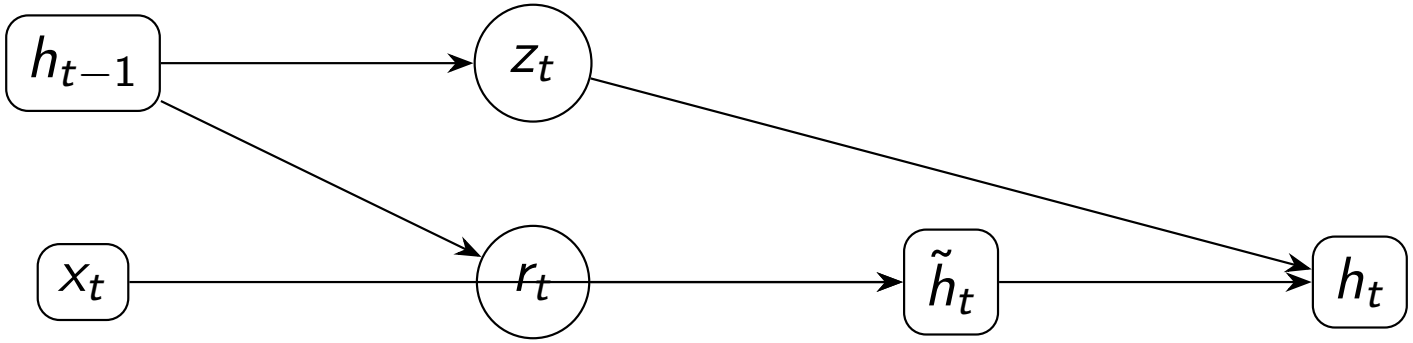
Key Components:

- Forget gate f_t
- Input gate i_t
- Candidate memory $\tilde{c}_t$
- Cell state c_t
- Output gate o_t

LSTM Cell (Conceptual Diagram)



GRU Cell (Conceptual Diagram)



$$h_t = (1 - z_t)\tilde{h}_t + z_t h_{t-1}$$

LSTM Gate and State Equations

Forget gate:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

Input gate:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

Candidate:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

Cell update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Output gate:

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

Hidden state:

$$h_t = o_t \odot \tanh(c_t)$$

How LSTMs Solve Vanishing Gradients

Cell-state recurrence:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t$$

Key derivative:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

If $f_t \approx 1$, then:

$$\frac{\partial c_t}{\partial c_{t-k}} \approx 1$$

Thus:

- Gradients flow without shrinking
- Memory persists over long sequences
- LSTM avoids exponential decay

This is the **Constant Error Carousel (CEC)**.

Numerical Example

Given:

$$x_t = 1, \; h_{t-1} = 0.5, \; c_{t-1} = 0.1$$

Weights all $= 1$.
Compute:
Forget:

$$f_t = \sigma(1.5) = 0.817$$

Input:

$$i_t = 0.817$$

Candidate:

$$\tilde{c}_t = \tanh(1.5) = 0.905$$

Cell:

$$c_t = 0.817(0.1) + 0.817(0.905) = 0.8207$$

Output:

$$h_t = 0.817 \cdot \tanh(0.8207) = 0.551$$

RNN vs LSTM vs GRU

	RNN	**LSTM**	**GRU**
Memory Type	Short-term	Long-term via c_t	Gated memory
Gates	None	3 gates	2 gates
Vanishing Gradients	Severe	Solved	Mostly solved
Complexity	Low	High	Medium
Usage	Simple seq.	Language, speech, seq2seq	General seq. tasks

Exercises: Conceptual

1. **Explain each LSTM gate:** $f_t, i_t, \tilde{c}_t, o_t$.
2. **Gradient behavior:**
   - Why do RNN gradients vanish?
   - Show mathematically how the LSTM cell state avoids vanishing.
3. **Intuition:**
   - What happens if $f_t = 0$?
   - What happens if $f_t = 1$ and $i_t = 0$?

Exercises: Numerical

Given:

$$x_t = 0.5,\ h_{t-1} = 0.2,\ c_{t-1} = 0.1$$

Weights:

$$W_f = 2,\ U_f = -1;\ W_i = 1,\ U_i = 1;\ W_c = 1,\ U_c = 0;\ W_o = -1,\ U_o = 2$$

Compute:

- $f_t, i_t, \tilde{c}_t$
- c_t, o_t, h_t

Practice: Identify Sequence Types

For each task, state the sequence type:

1. Predicting if a customer will churn based on a sequence of monthly usage data.
2. Generating an audio waveform from a single style vector.
3. Tagging each word of a sentence with its POS tag.
4. Translating an English sentence to French.

Solution: Sequence Types

1. Many-to-One
2. One-to-Many
3. Many-to-Many (aligned)
4. Many-to-Many (encoder–decoder)

References

Fleuret, F. (2024).
The Little Book of Deep Learning.
University of Geneva.

Goodfellow, I., Bengio, Y., Courville, A. (2016).
Deep Learning.
MIT Press, Cambridge.

Bishop, C. M., & Bishop, H. (2023).
Deep Learning: Foundations and Concepts.
Springer Nature.

Prince, S. J. (2023).
Understanding Deep Learning.
MIT Press.